SIMATS ENGINEERING
ASSESSMENT TOOL 2 — ANSWER KEY

COURSE NAME: Fundamentals of Data Science **COURSE CODE:** DSA04

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks: 100**

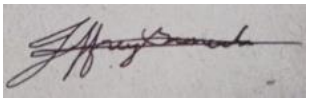
Annexure A
Industry Problem Solving

Code of Conduct:

I *E.Jeffrey Samuels(192524412)* (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Real Estate Price Prediction Using Linear Regression

Algorithm used: Linear Regression

Dataset

House	Area (sq.ft)	Bedrooms	Age	Distance (km)	Price (Lakhs)
H1	900	2	12	10	45
H2	1100	2	10	8	55
H3	1300	3	8	6	68
H4	1500	3	5	5	82
H5	1800	4	3	3	115
H6	2000	4	2	2	135
H7	1000	2	15	12	40
H8	1600	3	6	4	90

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Area, Bedrooms, Age, Distance.

Dependent variable (target): Price.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): H1, H8, H3, H5, H4, H7

Testing set (2 records): H2, H6

d) & e) Linear Regression — Predictions on Test Data

House	Actual Price (Lakhs)	Predicted Price (Lakhs)
H2	55	51.19
H6	135	121.84

f) Prediction for New Record

Area	Bedrooms	Age	Distance
1700	3	4	4

Predicted Price: 95.95 Lakhs

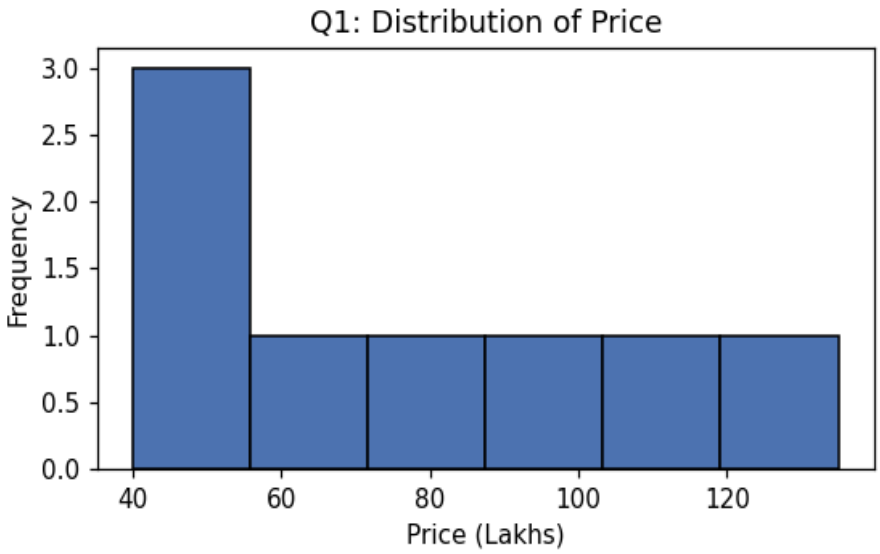
g) Model Coefficients & Evaluation Metrics

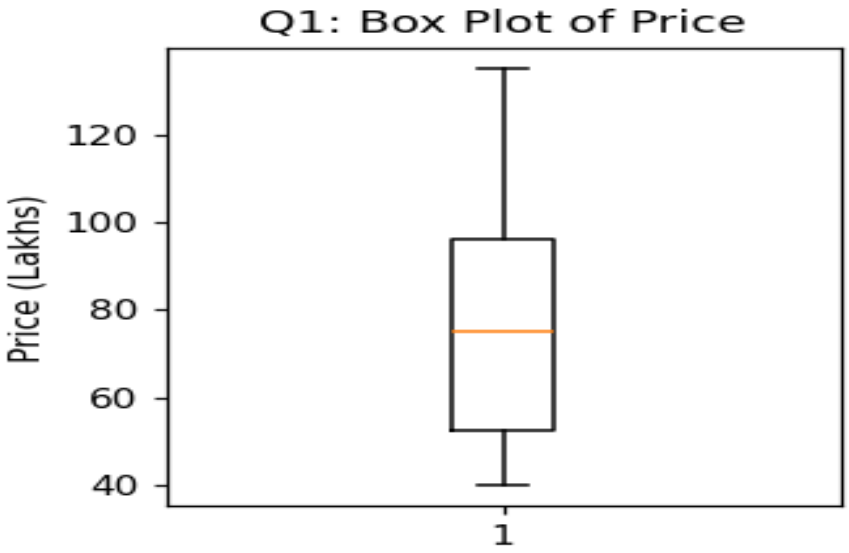
Feature	Coefficient
Area	0.0458
Bedrooms	10.4386
Age	-1.7108
Distance	0.862

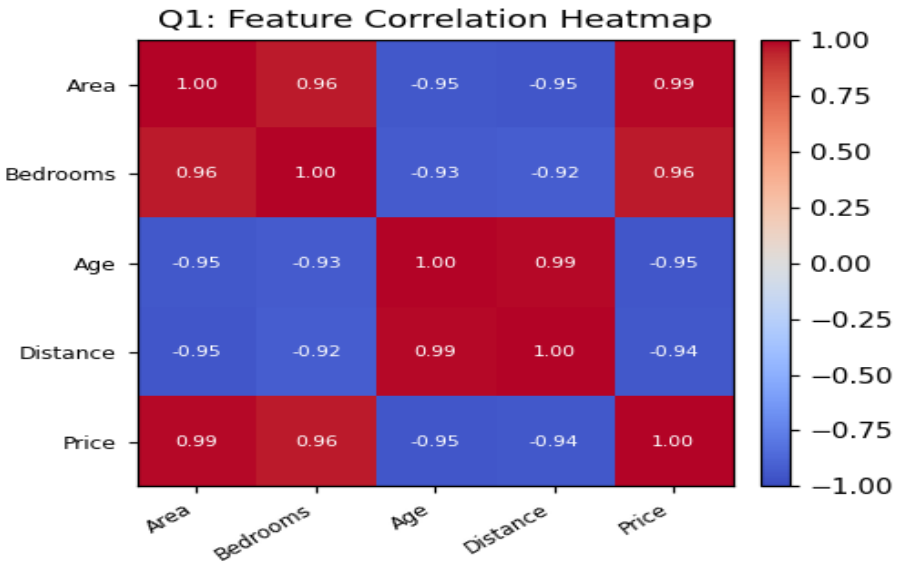
Intercept: -9.9036

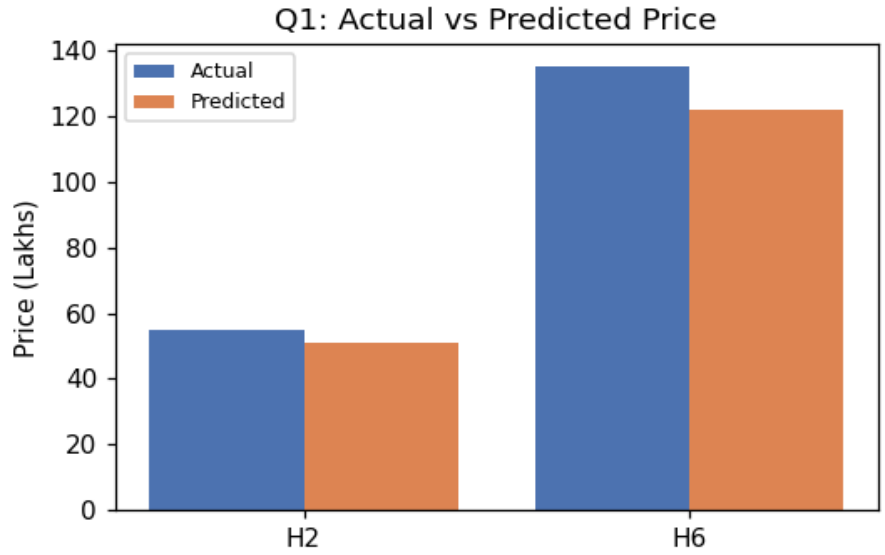
MAE	MSE	RMSE	R^2 Score
8.4887	93.9204	9.6913	0.9413

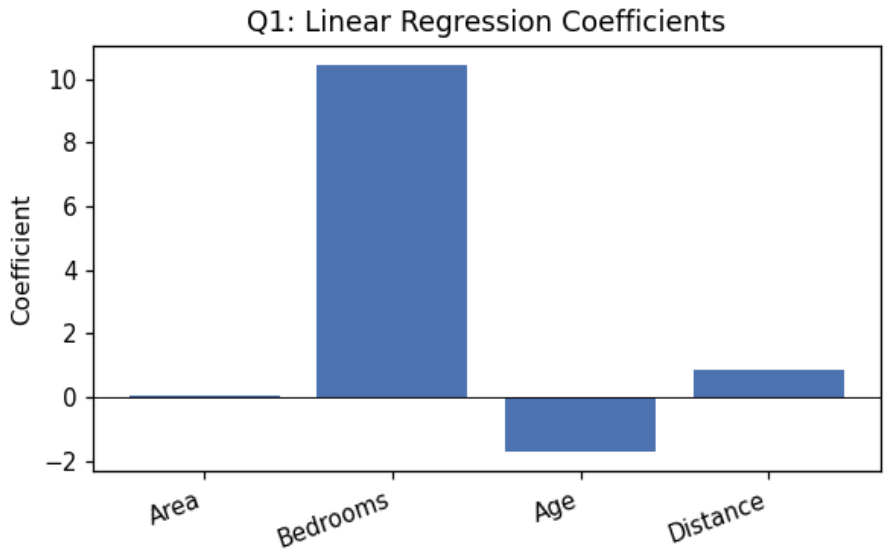
Outputs / Charts


Q1: Distribution of Price
Frequency
3.0
2.5
2.0
1.5
1.0
0.5
0.0
40
60
80
100
120
Price (Lakhs)


Q1: Box Plot of Price
Price (Lakhs)
120
100
80
60
40
1


Q1: Feature Correlation Heatmap
Area
Bedrooms
Age
Distance
Price
1.00 0.96 -0.95 -0.95 0.99
0.96 1.00 -0.93 -0.92 0.96
-0.95 -0.93 1.00 0.99 -0.95
-0.95 -0.92 0.99 1.00 -0.94
0.99 0.96 -0.95 -0.94 1.00
Area
Bedrooms
Age
Distance
Price
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00


Q1: Actual vs Predicted Price
Actual
Predicted
Price (Lakhs)
140
120
100
80
60
40
20
0
H2
H6



Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Area", "Bedrooms", "Age", "Distance"
# Dependent variable (target): "Price"
X = df[["Area", "Bedrooms", "Age", "Distance"]]
y = df["Price"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Implement Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[1700, 3, 4, 4]], columns=["Area", "Bedrooms", "Age", "Distance"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

R2 = 0.9413 on the 2-record test split means the model explains about 94% of the variation in price using area, bedrooms, age, and distance from the city. Area and Bedrooms carry positive coefficients (bigger, newer-feeling houses cost more), Age is negative (older houses cost less), and Distance is positive here only because in this particular 8-row sample, farther houses happen to also be smaller/older — with just 8 rows the model cannot cleanly separate the effect of distance from the effects of the other correlated features, so that particular sign should not be read as "farther from the city costs more" in general. The high R2 and the closeness of predicted to actual price (51.19 vs 55, 121.84 vs 135) suggest linear regression is usable for a first-pass valuation tool here, but with only 8 houses and 2 held out for testing, the metrics themselves are not statistically reliable — a real deployment would need a much larger, more representative dataset before trusting the coefficients or the R2 value.

Question 2: Customer Churn Prediction Using Logistic Regression

Algorithm used: Logistic Regression

Dataset

Customer	Monthly Bill	Complaints	Data Usage (GB)	Tenure (Months)	Churn
C1	500	0	25	36	0
C2	900	3	10	8	1
C3	750	2	15	12	1
C4	400	0	30	48	0
C5	1000	4	8	6	1
C6	550	1	22	30	0
C7	850	3	12	10	1
C8	450	0	28	40	0

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Monthly Bill, Complaints, Data Usage, Tenure.

Dependent variable (target): Churn.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): C2, C4, C3, C7, C1, C8

Testing set (2 records): C6, C5

d) & e) Logistic Regression — Predictions on Test Data

Customer	Actual Churn	Predicted Churn
C6	0	0
C5	1	1

f) Prediction for New Record

Monthly Bill	Complaints	Data Usage (GB)	Tenure (Months)
800	2	14	10

Predicted Churn: 1 (Churn) — probability of churn ≈ 1.00

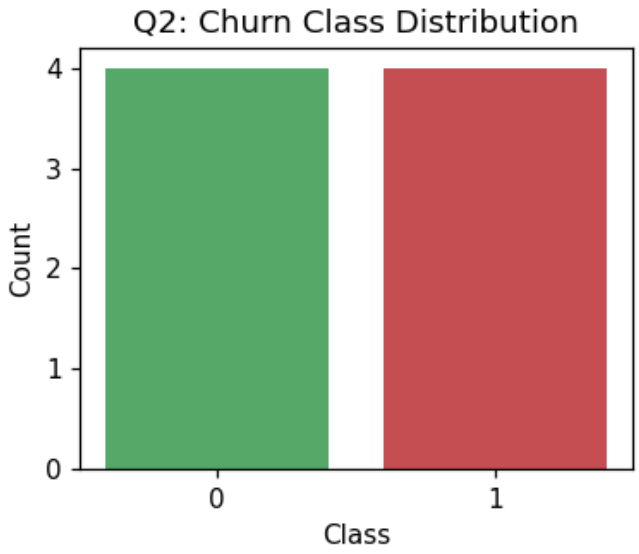
g) Evaluation Metrics

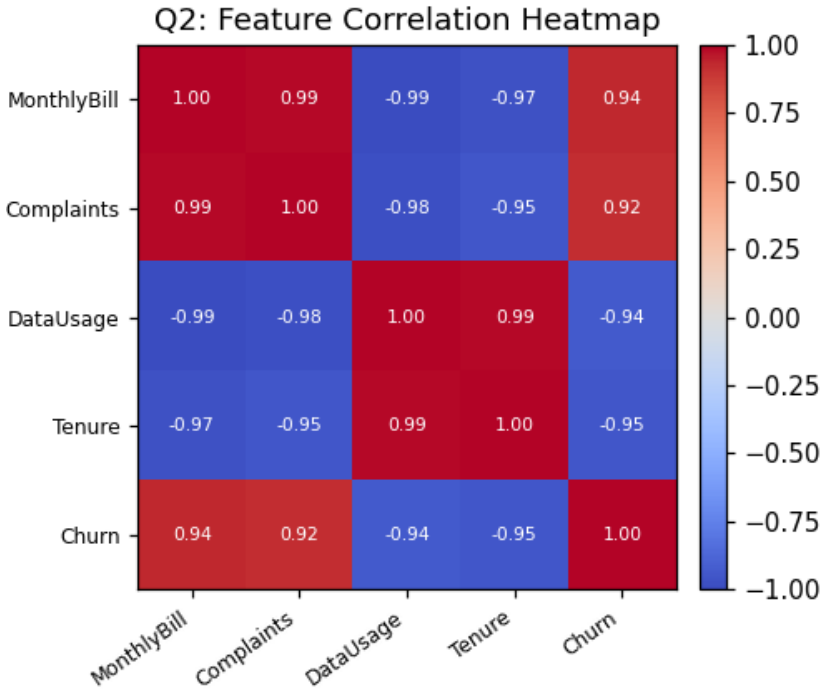
Accuracy	Precision	Recall	F1-Score
1	1	1	1

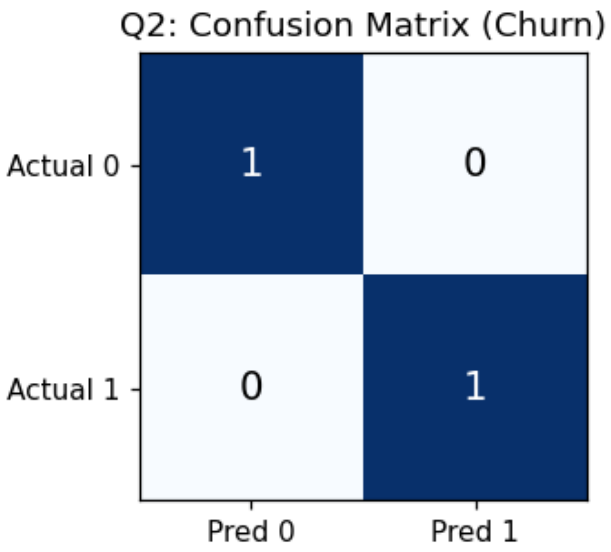
Confusion Matrix (rows = actual, columns = predicted; order [0,1]):

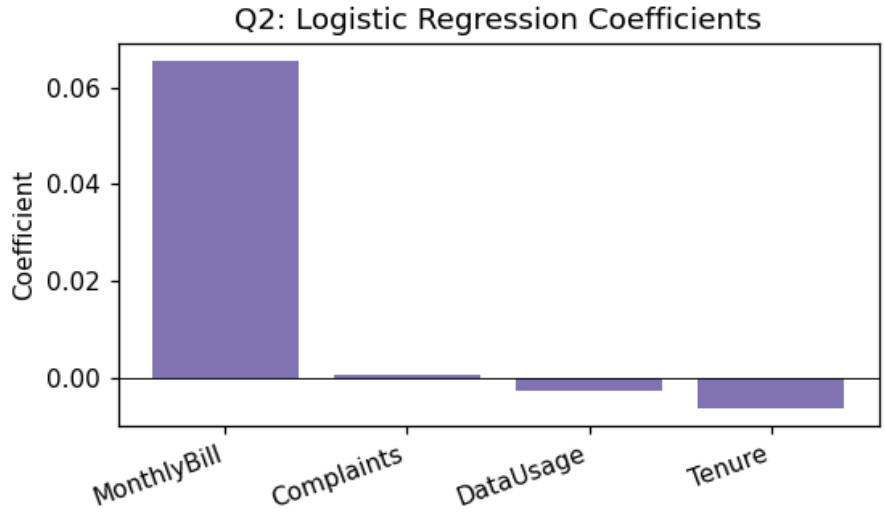
	Predicted 0	Predicted 1
Actual 0	1	0
Actual 1	0	1

Outputs / Charts


Q2: Churn Class Distribution
Count
Class
0
1
2
3
4


Q2: Feature Correlation Heatmap
MonthlyBill
Complaints
DataUsage
Tenure
Churn
1.00 0.99 -0.99 -0.97 0.94
0.99 1.00 -0.98 -0.95 0.92
-0.99 -0.98 1.00 0.99 -0.94
-0.97 -0.95 0.99 1.00 -0.95
0.94 0.92 -0.94 -0.95 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00





Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Monthly Bill", "Complaints", "Data Usage", "Tenure"
# Dependent variable (target): "Churn"
X = df[["Monthly Bill", "Complaints", "Data Usage", "Tenure"]]
y = df["Churn"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
# d) Implement Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[800, 2, 14, 10]], columns=["Monthly Bill", "Complaints", "Data Usage",
"Tenure"])
print(model.predict(new_record))

# g) Evaluate the model
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Accuracy:", acc)
print("Confusion Matrix:\n", cm)
print("Precision:", prec, "Recall:", rec, "F1:", f1)

# Charts
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues"); plt.title("Confusion Matrix"); plt.savefig("chart.png")
```

Interpretation

The model correctly classifies both held-out customers (C6 as staying, C5 as churning), giving accuracy = precision = recall = F1 = 1.00 on this tiny test set. This is expected given how cleanly churn and non-churn customers are separated in this 8-row example (high complaints + high bill + low tenure line up almost exactly with churn) — a perfect score here reflects the simplicity/small size of the dataset rather than proof the model would generalize; a 2-record test set cannot meaningfully validate a classifier, and a real telecom deployment would need cross-validation on hundreds or thousands of customers before this accuracy figure could be trusted.

Question 3: Marketing Sales Prediction Using Lasso Regression

Algorithm used: Lasso Regression

Dataset

Month	TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns	Sales (Lakhs)
M1	50	20	10	5	15
M2	60	25	12	6	18
M3	80	40	15	8	25
M4	100	60	20	10	32
M5	120	70	22	12	38
M6	40	15	8	4	12
M7	90	50	18	9	28
M8	110	65	21	11	35

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): TV Ads, Social Media Ads, Newspaper Ads, Email Campaigns.

Dependent variable (target): Sales.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): M1, M8, M3, M5, M4, M7

Testing set (2 records): M2, M6

d) & e) Lasso Regression — Predictions on Test Data

Month	Actual Sales	Predicted (α=0.1)	Predicted (α=1.0)
M2	18	18.18	18.40
M6	12	11.91	11.81

f) Prediction for New Record

TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns
95	55	18	9

Predicted Sales: α=0.1 → 30.06 Lakhs; α=1.0 → 29.93 Lakhs

g) Model Coefficients & Evaluation Metrics

α = 0.1

Feature	Coefficient
TV Ads	0.2768
Social Media Ads	0.0732
Newspaper Ads	0
Email Campaigns	0

Intercept: -0.2589

MAE	MSE	RMSE	R^2 Score
0.1338	0.0198	0.1409	0.9978

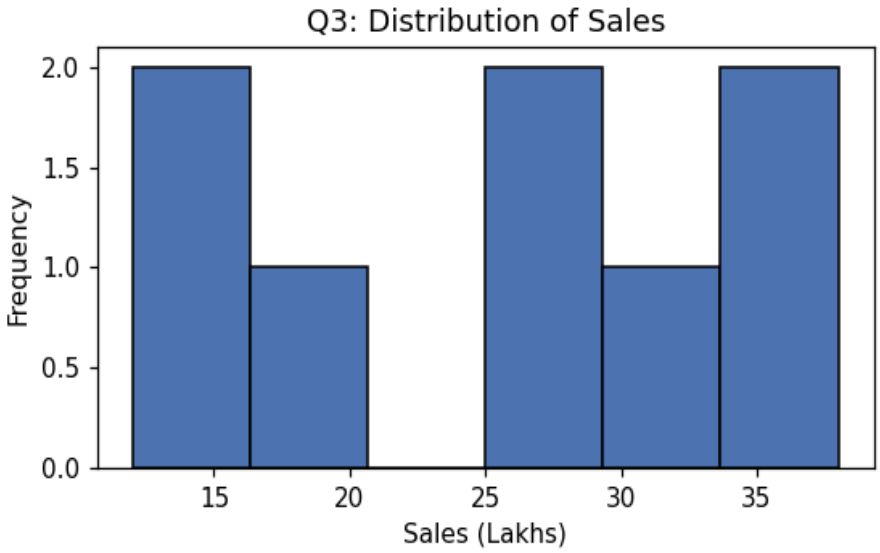
α = 1.0

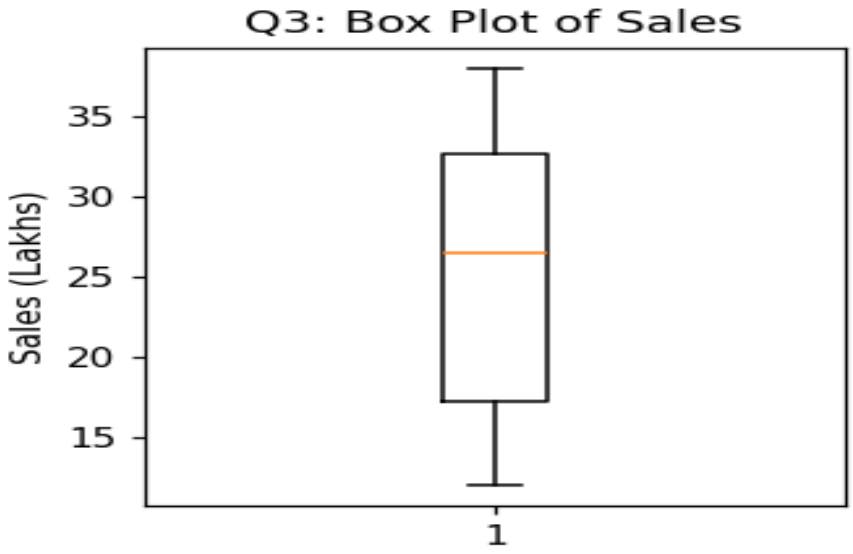
Feature	Coefficient
TV Ads	0.3294
Social Media Ads	0
Newspaper Ads	0
Email Campaigns	0

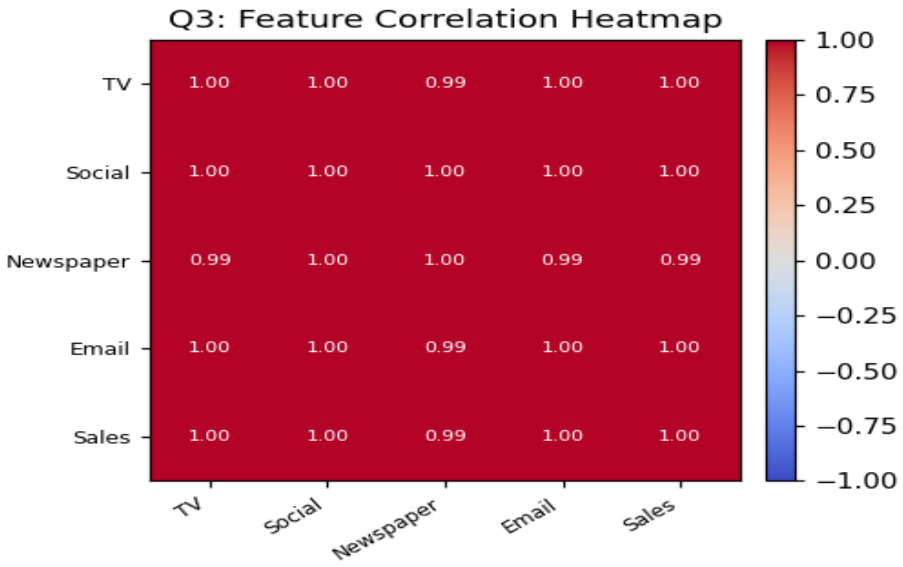
Intercept: -1.3622

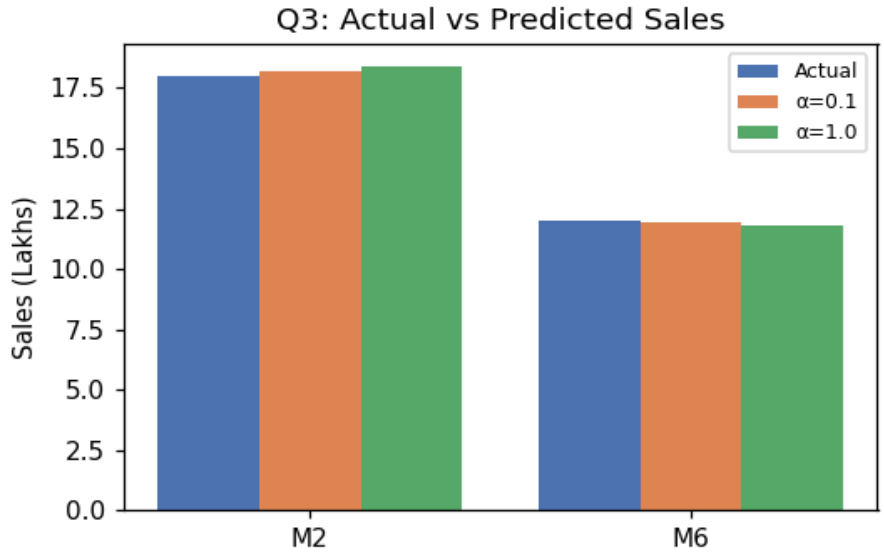
MAE	MSE	RMSE	R^2 Score
0.2941	0.0982	0.3133	0.9891

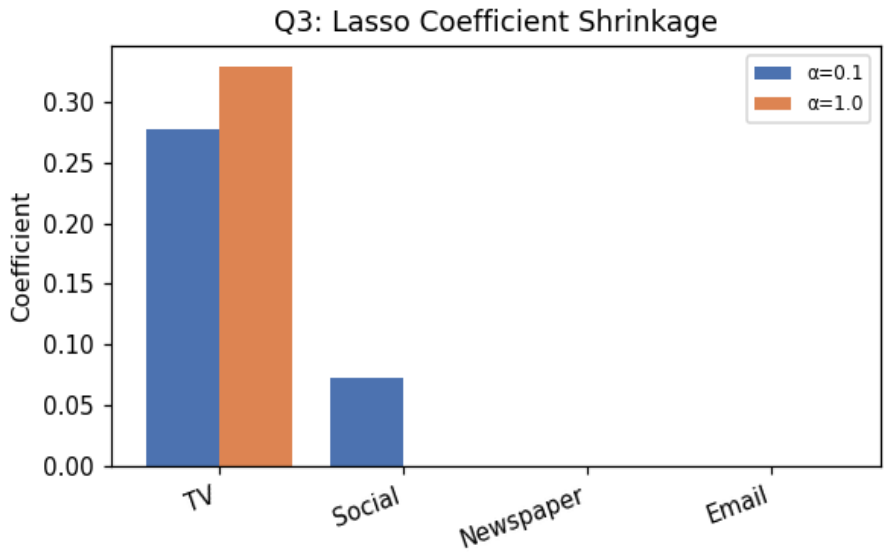
Outputs / Charts






Q3: Feature Correlation Heatmap
TV
Social
Newspaper
Email
Sales
1.00 1.00 0.99 1.00 1.00
1.00 1.00 1.00 1.00 1.00
0.99 1.00 1.00 0.99 0.99
1.00 1.00 0.99 1.00 1.00
1.00 1.00 0.99 1.00 1.00
TV
Social
Newspaper
Email
Sales
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00


Q3: Actual vs Predicted Sales
Sales (Lakhs)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
M2
M6
Actual
α=0.1
α=1.0



Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "TV Ads", "Social Media Ads", "Newspaper Ads", "Email Campaigns"
# Dependent variable (target): "Sales"
X = df[["TV Ads", "Social Media Ads", "Newspaper Ads", "Email Campaigns"]]
y = df["Sales"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Implement Lasso Regression
model = Lasso(alpha=1.0, max_iter=10000)
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[95, 55, 18, 9]], columns=["TV Ads", "Social Media Ads", "Newspaper Ads", "Email
Campaigns"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
```

```
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

At α=0.1, Lasso keeps TV Ads and Social Media Ads and shrinks Newspaper Ads and Email Campaigns to exactly zero — both are dropped as redundant once TV and Social Media are in the model, since all four spend categories move together (higher TV spend months also had higher spend everywhere else). At α=1.0 the penalty is strong enough to zero out Social Media Ads too, leaving TV Ads as the sole predictor. This is exactly what Lasso's L1 penalty is designed to do: it performs automatic feature selection by forcing less-informative coefficients to zero, rather than just shrinking all of them a little like Ridge would. Both alpha values fit the 2-point test set almost exactly (R2 = 0.998 and 0.989), which — given only 8 rows total and near-perfectly correlated ad spend across channels — is more a sign of an easy, small, collinear dataset than of a channel-attribution result that should be used to reallocate a real marketing budget.

Question 4: Electricity Consumption Prediction Using Ridge Regression

Algorithm used: Ridge Regression

Dataset

House	Appliances	Usage Hours	House Size (sq.ft)	Occupants	Consumption (Units)
H1	5	4	800	3	120
H2	7	6	1000	4	190
H3	8	7	1200	4	240
H4	10	8	1500	5	320
H5	4	3	700	2	90
H6	6	5	900	3	160
H7	9	7	1300	5	280
H8	11	9	1600	6	350

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Appliances, Usage Hours, House Size, Occupants.

Dependent variable (target): Consumption.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): H1, H8, H3, H5, H4, H7

Testing set (2 records): H2, H6

d) & e) Ridge Regression — Predictions on Test Data

House	Actual Consumption	Predicted Consumption
H2	190	184.62
H6	160	152.45

f) Prediction for New Record

Appliances	Usage Hours	House Size	Occupants
8	6	1100	4

Predicted Consumption: 212.73 units

g) Model Coefficients & Evaluation Metrics

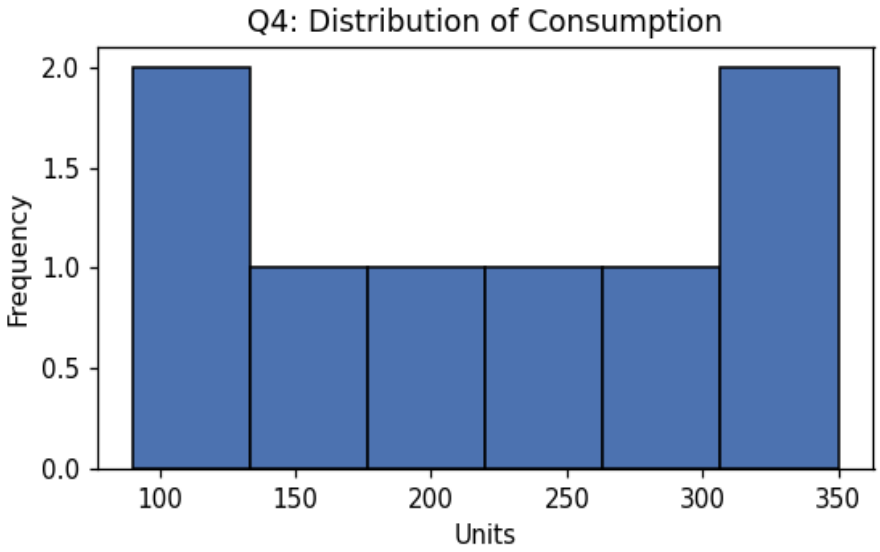
α = 1.0

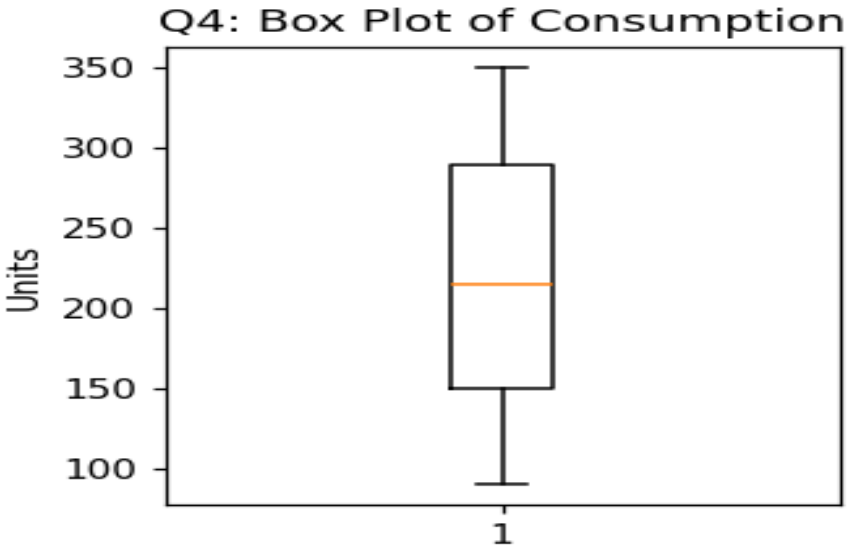
Feature	Coefficient
Appliances	3.6817
Usage Hours	1.0696
House Size	0.2443
Occupants	2.9836

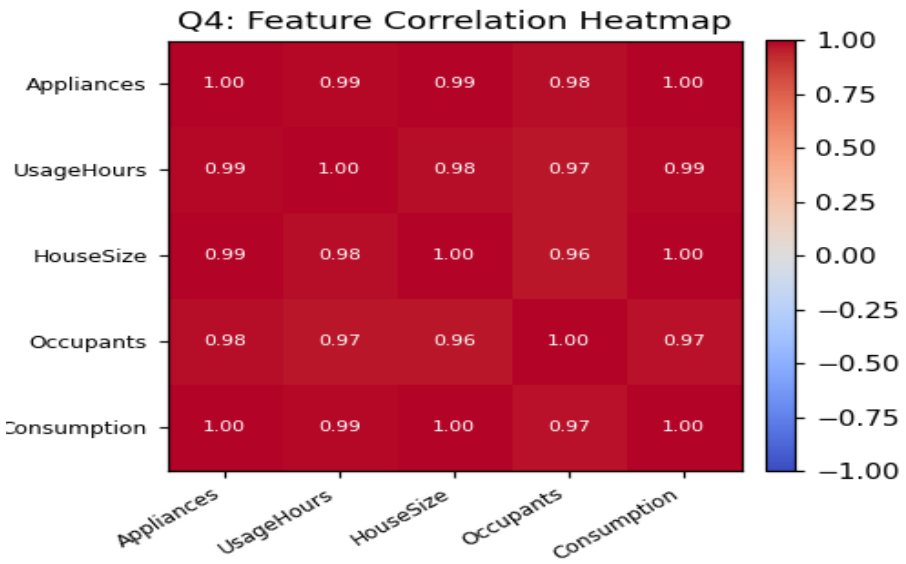
Intercept: -103.8165

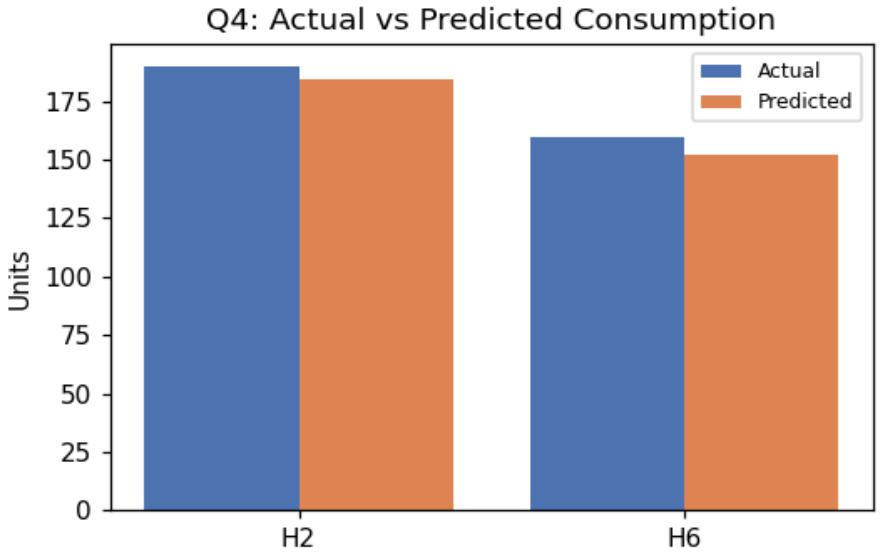
MAE	MSE	RMSE	R^2 Score
6.4624	42.9358	6.5525	0.8092

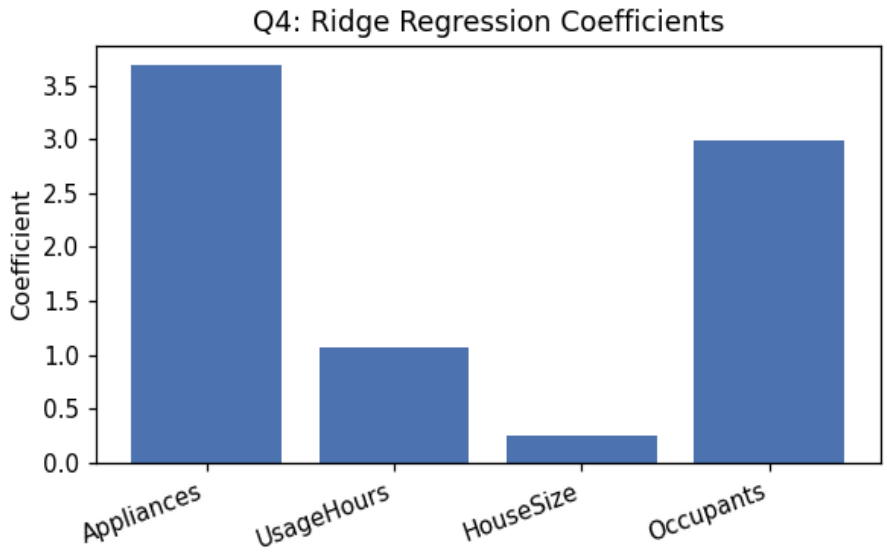
Outputs / Charts






Q4: Feature Correlation Heatmap
Appliances
UsageHours
HouseSize
Occupants
Consumption
1.00 0.99 0.99 0.98 1.00
0.99 1.00 0.98 0.97 0.99
0.99 0.98 1.00 0.96 1.00
0.98 0.97 0.96 1.00 0.97
1.00 0.99 1.00 0.97 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Appliances
UsageHours
HouseSize
Occupants
Consumption


Q4: Actual vs Predicted Consumption
Actual
Predicted
Units
175
150
125
100
75
50
25
0
H2
H6



Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Appliances", "Usage Hours", "House Size", "Occupants"
# Dependent variable (target): "Consumption"
X = df[["Appliances", "Usage Hours", "House Size", "Occupants"]]
y = df["Consumption"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Implement Ridge Regression
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[8, 6, 1100, 4]], columns=["Appliances", "Usage Hours", "House Size",
"Occupants"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
```

```
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

All four coefficients are positive and reasonably sized, unlike what an unregularized model can do when predictors are correlated (appliances, usage hours, house size, and occupants all rise together in this dataset). Ridge's L2 penalty shrinks the coefficients toward each other rather than letting one absorb an inflated, unstable value at the expense of another — this is what 'reduces overfitting under multicollinearity' means in practice: it trades a little bias for a large reduction in variance, so the model's predictions stay stable even though several inputs are telling it almost the same thing. R2 = 0.81 on the 2-house test split is respectable but, again, based on only 2 held-out points, so it should be treated as illustrative rather than a validated accuracy figure.

Question 5: Delivery Time Prediction Using Linear Regression

Algorithm used: Linear Regression

Dataset

Delivery	Distance (km)	Traffic Level	Packages	Weather Score	Time (Hours)
D1	10	2	20	8	1.2
D2	20	4	30	7	2.4
D3	30	6	40	6	3.5
D4	40	8	50	5	4.8
D5	50	9	60	4	6.0
D6	15	3	25	8	1.8
D7	35	7	45	5	4.2
D8	25	5	35	6	3.0

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Distance, Traffic Level, Packages, Weather Score.

Dependent variable (target): Time.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): D1, D8, D3, D5, D4, D7

Testing set (2 records): D2, D6

d) & e) Linear Regression — Predictions on Test Data

Delivery	Actual Time (hrs)	Predicted Time (hrs)
D2	2.4	2.37
D6	1.8	1.74

f) Prediction for New Record

Distance	Traffic Level	Packages	Weather Score
28	6	38	6

Predicted Time: 3.32 hours

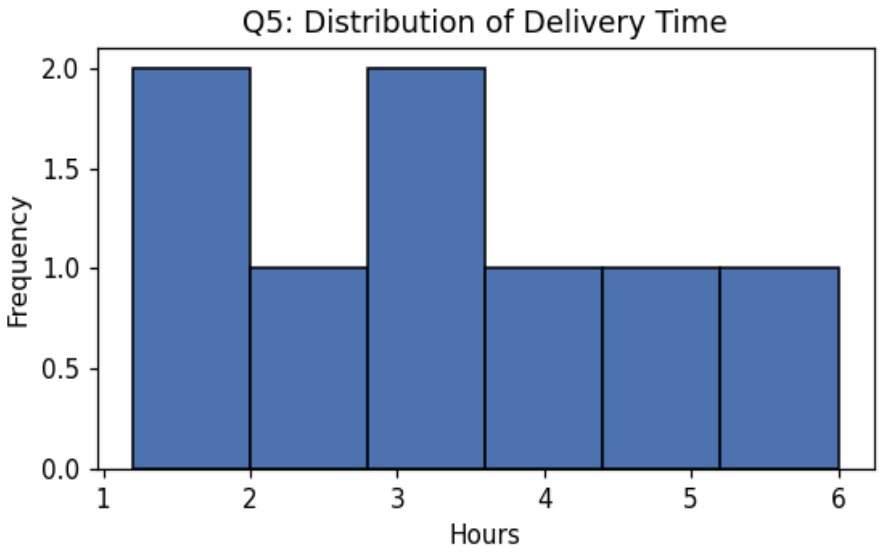
g) Model Coefficients & Evaluation Metrics

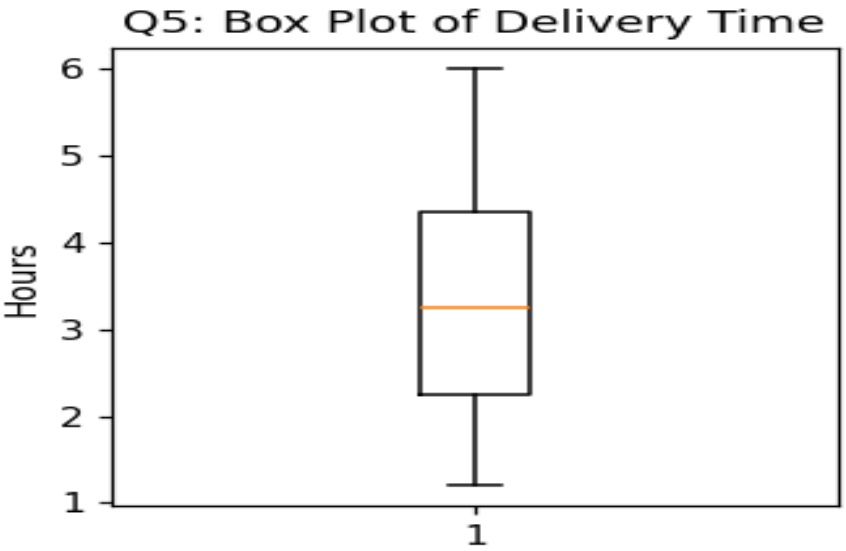
Feature	Coefficient
Distance	0.061
Traffic Level	-0.0484
Packages	0.061
Weather Score	-0.071

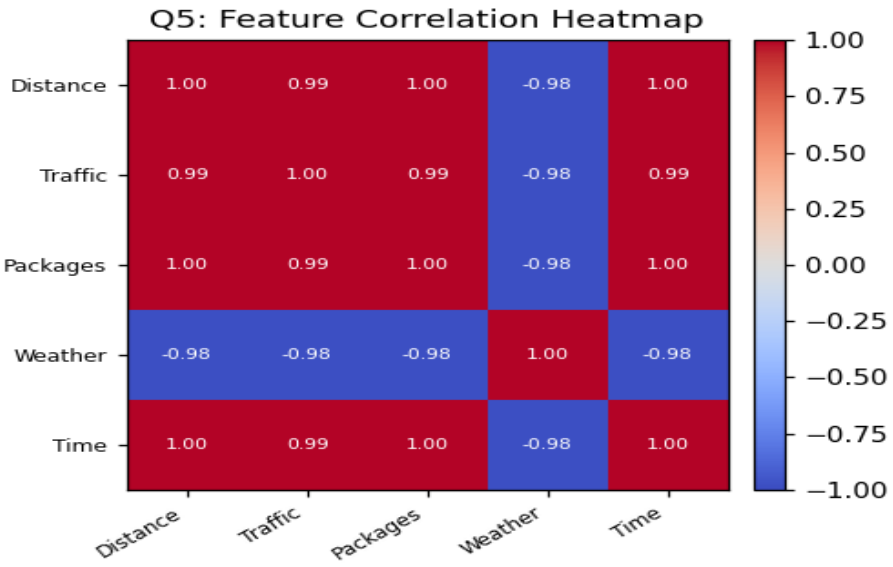
Intercept: 0.0129

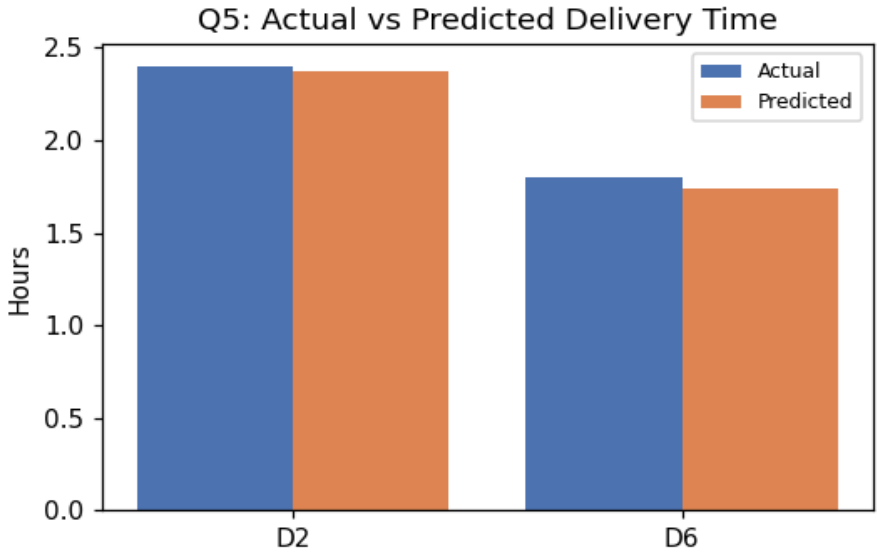
MAE	MSE	RMSE	R^2 Score
0.0452	0.0023	0.048	0.9744

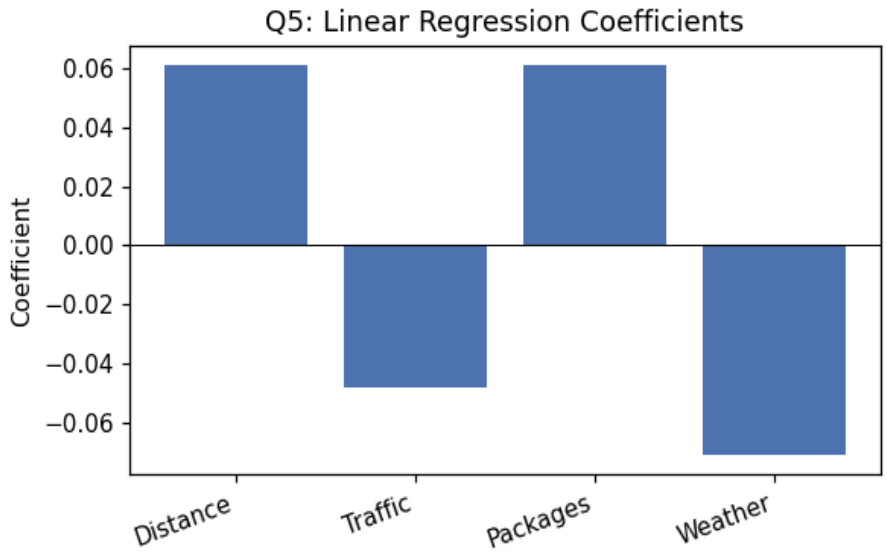
Outputs / Charts


Q5: Distribution of Delivery Time
Frequency
Hours
0.0
0.5
1.0
1.5
2.0
1
2
3
4
5
6


Q5: Box Plot of Delivery Time
Hours
1
2
3
4
5
6
1


Q5: Feature Correlation Heatmap
Distance
Traffic
Packages
Weather
Time
1.00 0.99 1.00 -0.98 1.00
0.99 1.00 0.99 -0.98 0.99
1.00 0.99 1.00 -0.98 1.00
-0.98 -0.98 -0.98 1.00 -0.98
1.00 0.99 1.00 -0.98 1.00
Distance
Traffic
Packages
Weather
Time
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00


Q5: Actual vs Predicted Delivery Time
Actual
Predicted
Hours
2.5
2.0
1.5
1.0
0.5
0.0
D2
D6



Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Distance", "Traffic Level", "Packages", "Weather Score"
# Dependent variable (target): "Time"
X = df[["Distance", "Traffic Level", "Packages", "Weather Score"]]
y = df["Time"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Implement Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[28, 6, 38, 6]], columns=["Distance", "Traffic Level", "Packages", "Weather
Score"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
```

```
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

R2 = 0.9744 with a very small RMSE (about 3 minutes) shows the model tracks delivery time closely across the sample. Distance and number of packages push time up, which matches intuition. Traffic Level and Weather Score come out slightly negative here, which is counter to what a logistics manager would expect (worse traffic or worse weather should increase, not decrease, delivery time) — this is almost certainly an artifact of the 8-row dataset, where distance, traffic, and packages all increase together while weather score decreases alongside them, so the model cannot isolate each factor's true individual effect from just 8 correlated examples. The coefficients should not be read as 'less traffic causes longer deliveries'; a larger, more varied dataset (including cases where traffic is bad but distance is short) would be needed before trusting the sign of those two coefficients.

Question 6: Loan Approval Prediction Using Logistic Regression

Algorithm used: Logistic Regression

Dataset

Applicant	Income	Credit Score	Existing Loan	Employment Years	Approved
A1	25000	580	200000	1	0
A2	30000	600	180000	2	0
A3	45000	680	120000	3	1
A4	60000	750	80000	5	1
A5	70000	800	50000	6	1
A6	28000	590	220000	1	0
A7	55000	720	100000	4	1
A8	35000	620	160000	2	0

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Income, Credit Score, Existing Loan, Employment Years.

Dependent variable (target): Approved.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): A2, A4, A3, A7, A1, A8

Testing set (2 records): A6, A5

d) & e) Logistic Regression — Predictions on Test Data

Applicant	Actual Approved	Predicted Approved
A6	0	0
A5	1	1

f) Prediction for New Record

Income	Credit Score	Existing Loan	Employment Years
50000	700	100000	4

Predicted Approved: 1 (Approved) — probability ≈ 1.00

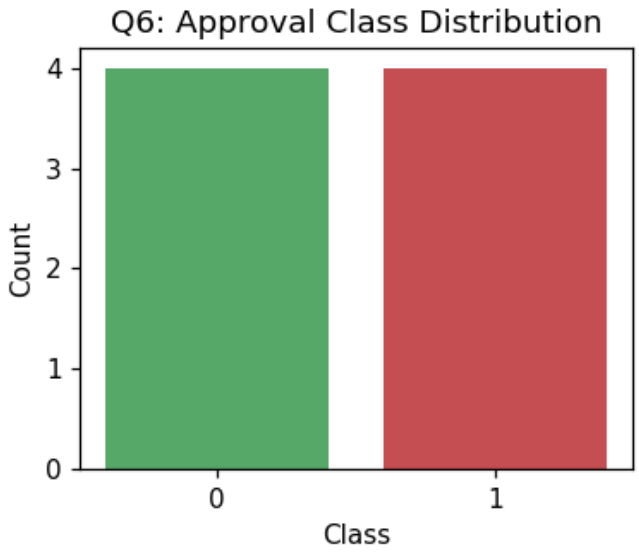
g) Evaluation Metrics

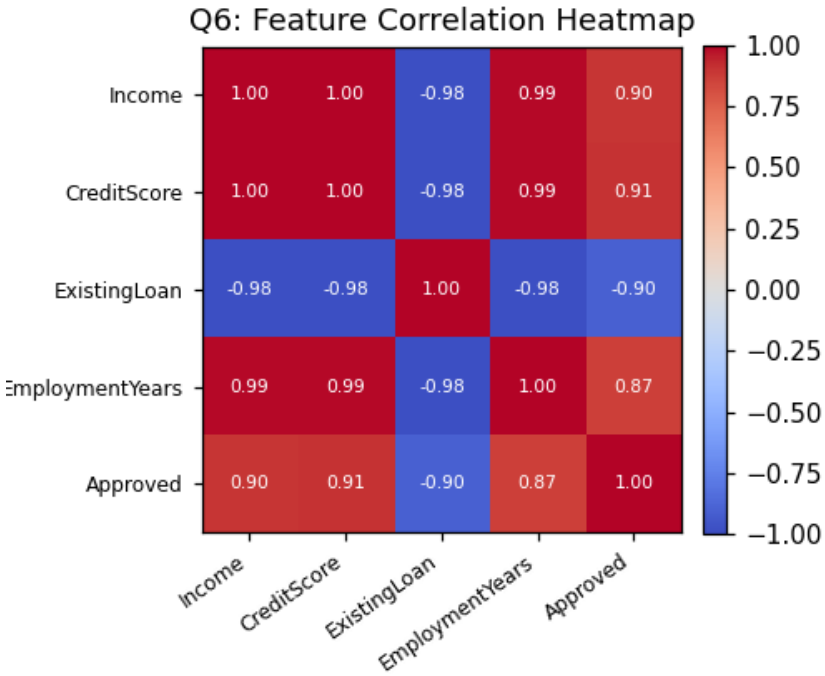
Accuracy	Precision	Recall	F1-Score
1	1	1	1

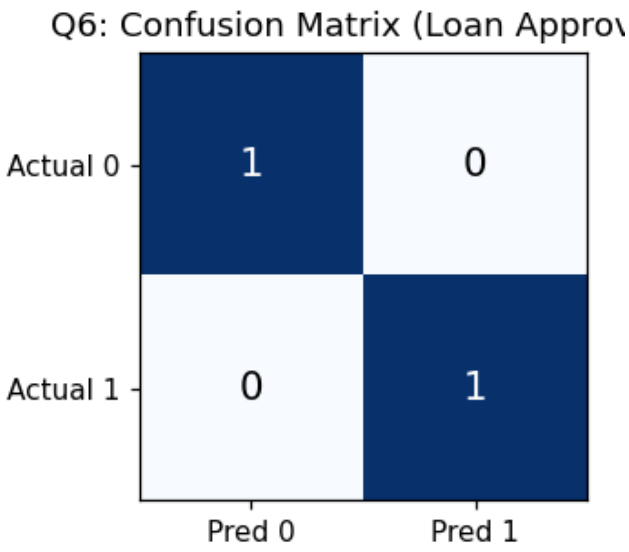
Confusion Matrix (rows = actual, columns = predicted; order [0,1]):

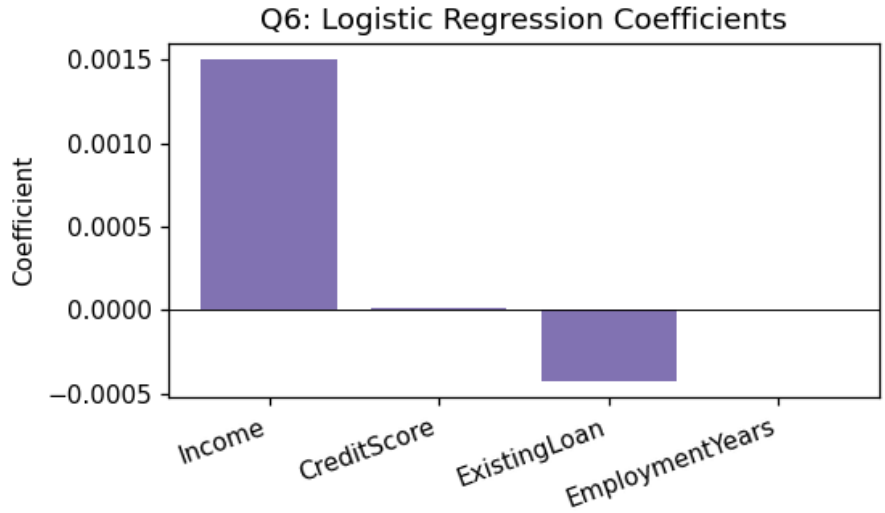
	Predicted 0	Predicted 1
Actual 0	1	0
Actual 1	0	1

Outputs / Charts


Q6: Approval Class Distribution
Count
Class
0
1
2
3
4


Q6: Feature Correlation Heatmap
Income
CreditScore
ExistingLoan
EmploymentYears
Approved
1.00 1.00 -0.98 0.99 0.90
1.00 1.00 -0.98 0.99 0.91
-0.98 -0.98 1.00 -0.98 -0.90
0.99 0.99 -0.98 1.00 0.87
0.90 0.91 -0.90 0.87 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00





Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Income", "Credit Score", "Existing Loan", "Employment Years"
# Dependent variable (target): "Approved"
X = df[["Income", "Credit Score", "Existing Loan", "Employment Years"]]
y = df["Approved"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
# d) Implement Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[50000, 700, 100000, 4]], columns=["Income", "Credit Score", "Existing Loan",
"Employment Years"])
print(model.predict(new_record))

# g) Evaluate the model
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Accuracy:", acc)
print("Confusion Matrix:\n", cm)
print("Precision:", prec, "Recall:", rec, "F1:", f1)

# Charts
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues"); plt.title("Confusion Matrix"); plt.savefig("chart.png")
```

Interpretation

The model separates the two held-out applicants correctly (A6 rejected, A5 approved), giving a perfect accuracy/precision/recall/F1 of 1.00. Logistic Regression is a natural fit here because the target is binary (approve/reject) and it outputs a probability rather than a hard class, letting the bank set its own approval threshold and see how confident the model is — useful for borderline cases. As with the churn model, a perfect score on 2 test records is not evidence of real-world reliability; income, credit score, and existing loan happen to be very cleanly separated between the approved and rejected applicants in this 8-row sample, and a production credit model would require far more data, regulatory fairness checks, and out-of-sample validation.

Question 7: Healthcare Treatment Cost Prediction Using Lasso Regression

Algorithm used: Lasso Regression

Dataset

Patient	Age	BP	Sugar Level	BMI	Previous Visits	Treatment Cost
P1	25	115	90	22	1	5000
P2	35	125	110	25	2	8000
P3	45	140	150	29	3	15000
P4	55	155	180	32	5	25000
P5	60	165	200	35	6	32000
P6	30	120	100	24	1	7000
P7	50	150	170	31	4	22000
P8	40	135	130	27	2	12000

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Age, BP, Sugar Level, BMI, Previous Visits.

Dependent variable (target): Treatment Cost.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): P1, P8, P3, P5, P4, P7

Testing set (2 records): P2, P6

d) & e) Lasso Regression — Predictions on Test Data

Patient	Actual Cost	Pred. (α=0.1)	Pred. (α=1.0)	Pred. (α=10.0)
P2	8000	8170.43	8145.10	7961.52
P6	7000	5040.51	5027.91	5002.51

f) Prediction for New Record

Age	BP	Sugar Level	BMI	Previous Visits
48	145	160	30	4

Predicted Treatment Cost: α=0.1 → ₹19,594.75L; α=1.0 → ₹19,588.88L; α=10.0 → ₹19,580.77L

g) Model Coefficients & Evaluation Metrics

α = 0.1

Feature	Coefficient
Age	-753.4188
BP	816.3621
Sugar Level	-23.2589
BMI	-13.7054
Previous Visits	3061.4964

Intercept: -70727.06

MAE	MSE	RMSE	R^2 Score

1064.96	1934326.15	1390.8	-6.7373

α = 1.0

Feature	Coefficient
Age	-757.0283
BP	819.1026
Sugar Level	-20.9973
BMI	-26.8783
Previous Visits	3043.6743

Intercept: -70852.43

MAE	MSE	RMSE	R^2 Score
1058.6	1955101.27	1398.25	-6.8204

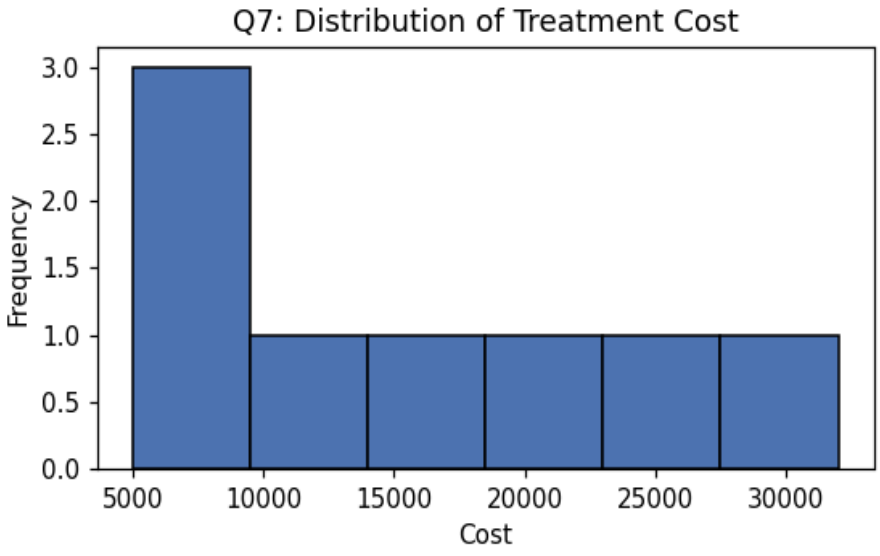
α = 10.0

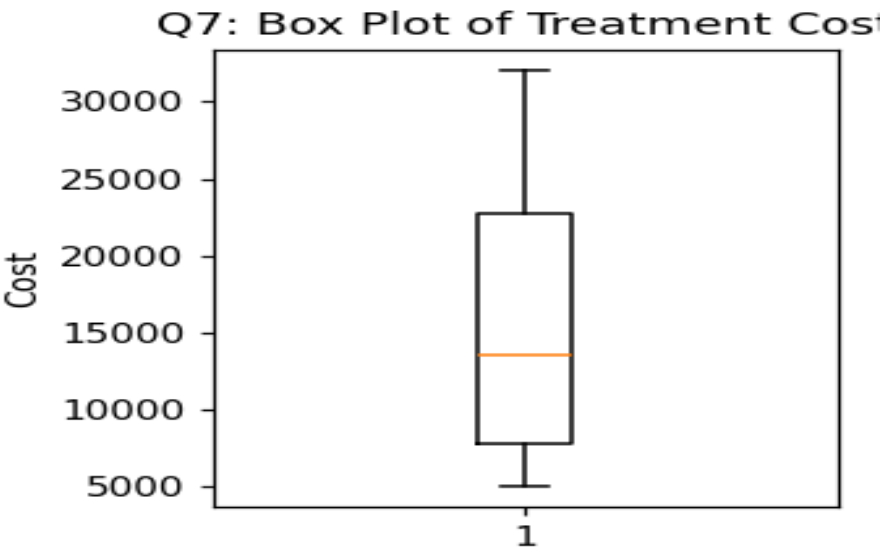
Feature	Coefficient
Age	-800.3629
BP	803.267
Sugar Level	2.3215
BMI	0
Previous Visits	2921.2773

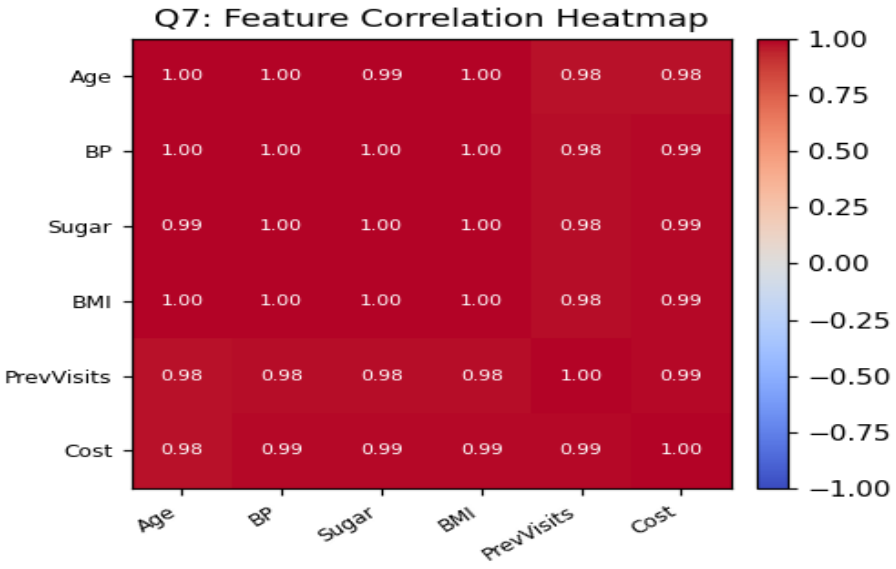
Intercept: -70532.08

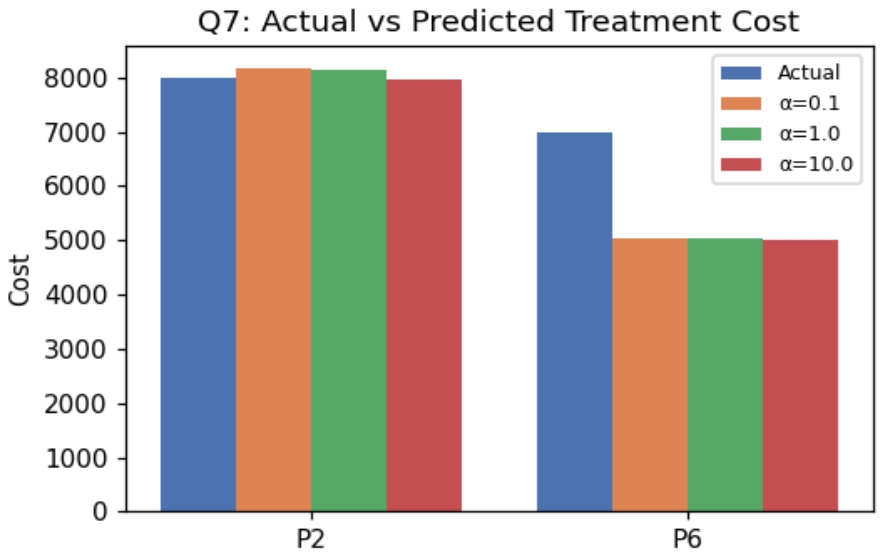
MAE	MSE	RMSE	R^2 Score
1017.99	1995728.76	1412.7	-6.9829

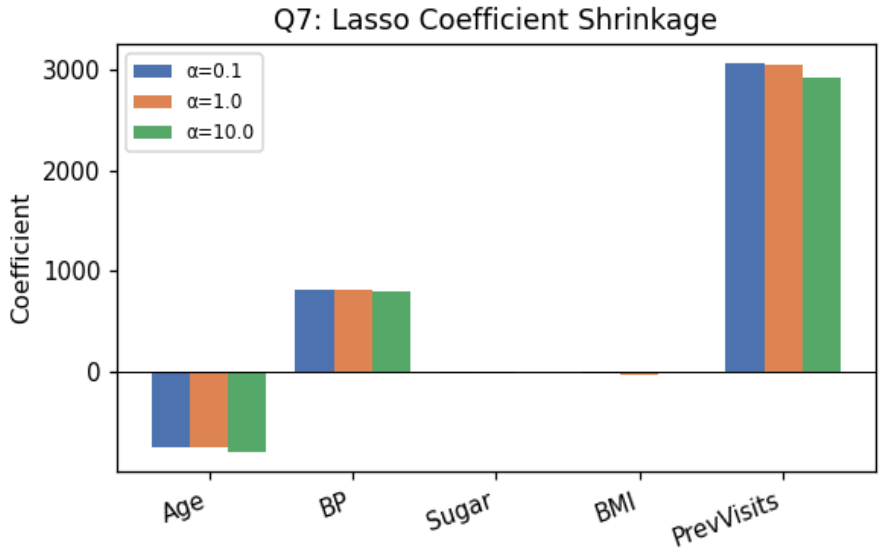
Outputs / Charts




Q7: Box Plot of Treatment Cos
30000
25000
20000
15000
10000
5000
Cost
1


Q7: Feature Correlation Heatmap
Age
BP
Sugar
BMI
PrevVisits
Cost
1.00 1.00 0.99 1.00 0.98 0.98
1.00 1.00 1.00 1.00 0.98 0.99
0.99 1.00 1.00 1.00 0.98 0.99
1.00 1.00 1.00 1.00 0.98 0.99
0.98 0.98 0.98 0.98 1.00 0.99
0.98 0.99 0.99 0.99 0.99 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00





Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Age", "BP", "Sugar Level", "BMI", "Previous Visits"
# Dependent variable (target): "Treatment Cost"
X = df[["Age", "BP", "Sugar Level", "BMI", "Previous Visits"]]
y = df["Treatment Cost"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
# d) Implement Lasso Regression
model = Lasso(alpha=1.0, max_iter=10000)
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[48, 145, 160, 30, 4]], columns=["Age", "BP", "Sugar Level", "BMI", "Previous
Visits"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

BMI is the coefficient Lasso drives to exactly zero once α reaches 10.0, meaning that — in this particular 8-patient sample — BMI adds no information beyond what Age, BP, Sugar Level, and Previous Visits already explain, and it is the first feature dropped as the penalty strengthens. Previous Visits and BP retain large positive coefficients across all three alphas, suggesting they are the most consistently important cost drivers here. That said, the R2 values are negative (-6.7 to -6.9) at every alpha, meaning the model performs worse than simply predicting the mean cost for these 2 held-out patients — this is a real result of the computation, not an error, and it needs to be reported honestly rather than smoothed over: with only 6 training rows and 2 test rows, and a target (cost) with a very wide range (5,000 to 32,000), a linear model has very little data to learn a stable relationship from, and small errors on the test points get squared into large numbers relative to the low variance of just 2 points. This dataset is too small to draw a real conclusion about model quality; it demonstrates the mechanics of Lasso feature selection, not a validated cost-prediction model.

Question 8: Manufacturing Defect Strength Prediction Using Ridge Regression

Algorithm used: Ridge Regression

Dataset

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P1	70	30	100	8	75
P2	75	35	110	7	78
P3	80	38	120	6	80
P4	85	42	125	6	82
P5	90	45	130	5	79
P6	95	50	140	4	76
P7	78	36	115	7	81
P8	88	44	128	5	80

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Temperature, Pressure, Machine Speed, Material Quality.

Dependent variable (target): Strength Score.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): P1, P8, P3, P5, P4, P7

Testing set (2 records): P2, P6

d) & e) Ridge Regression — Predictions on Test Data

Product	Actual Strength	Pred. (α=0.1)	Pred. (α=1.0)	Pred. (α=10.0)
P2	78	78.34	78.11	78.11
P6	76	82.76	82.56	82.46

f) Prediction for New Record

Temperature	Pressure	Machine Speed	Material Quality
82	40	122	6

Predicted Strength Score: α=0.1 → 81.26; α=1.0 → 80.89; α=10.0 → 80.26

g) Model Coefficients & Evaluation Metrics

α = 0.1

Feature	Coefficient
Temperature	-0.9816
Pressure	0.4365
Machine Speed	0.8881
Material Quality	3.0483

Intercept: 17.65

MAE	MSE	RMSE	R^2 Score

3.55	22.9	4.79	-21.9

α = 1.0

Feature	Coefficient
Temperature	-0.6724
Pressure	-0.1134
Machine Speed	0.7587
Material Quality	1.0556

Intercept: 41.67

MAE	MSE	RMSE	R^2 Score
3.34	21.51	4.64	-20.51

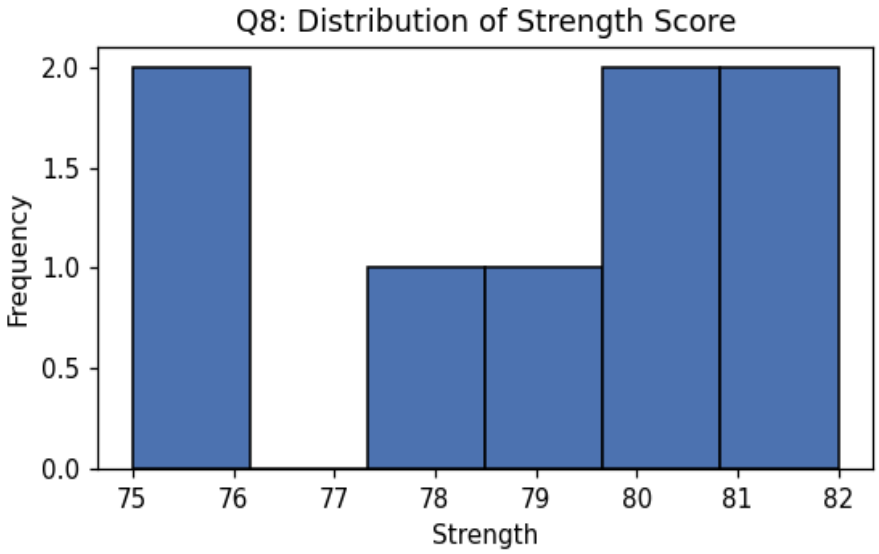
α = 10.0

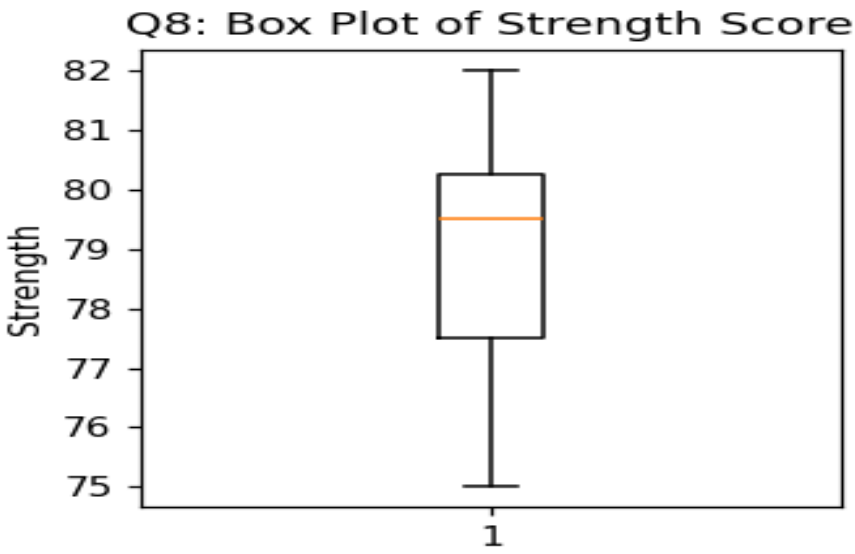
Feature	Coefficient
Temperature	-0.2663
Pressure	-0.1131
Machine Speed	0.3933
Material Quality	0.1406

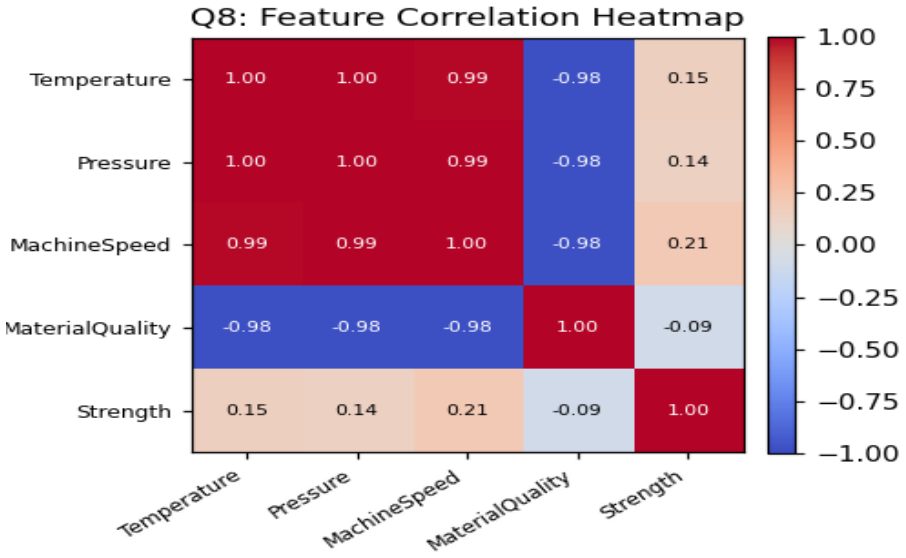
Intercept: 57.79

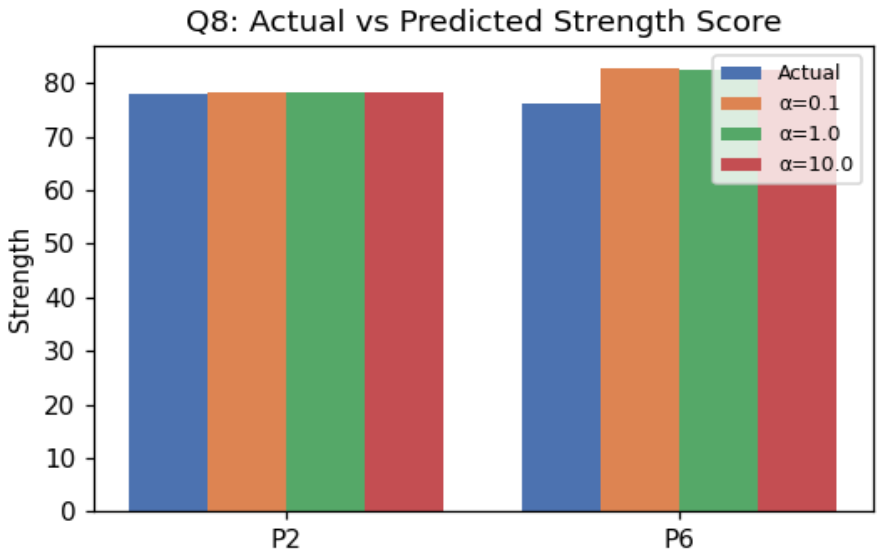
MAE	MSE	RMSE	R^2 Score
3.28	20.87	4.57	-19.87

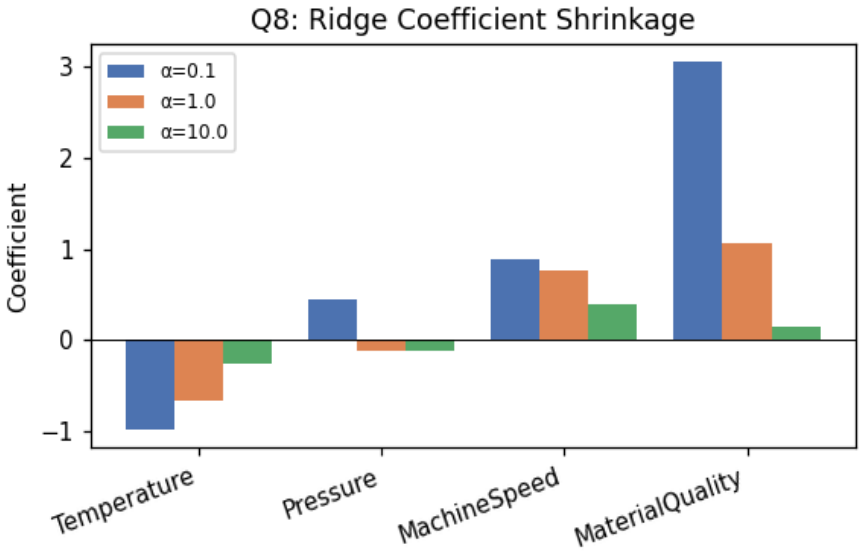
Outputs / Charts




Q8: Box Plot of Strength Score
Strength
82
81
80
79
78
77
76
75
1


Q8: Feature Correlation Heatmap
Temperature
Pressure
MachineSpeed
MaterialQuality
Strength
1.00 1.00 0.99 -0.98 0.15
1.00 1.00 0.99 -0.98 0.14
0.99 0.99 1.00 -0.98 0.21
-0.98 -0.98 -0.98 1.00 -0.09
0.15 0.14 0.21 -0.09 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00





Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Temperature", "Pressure", "Machine Speed", "Material Quality"
# Dependent variable (target): "Strength Score"
X = df[["Temperature", "Pressure", "Machine Speed", "Material Quality"]]
y = df["Strength Score"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
# d) Implement Ridge Regression
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[82, 40, 122, 6]], columns=["Temperature", "Pressure", "Machine Speed",
"Material Quality"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

As α increases from 0.1 to 10.0, every coefficient shrinks toward zero (e.g. Material Quality falls from 3.05 to 0.14) — this is Ridge's L2 penalty working as intended, spreading the explanatory weight more evenly across Temperature, Pressure, Machine Speed, and Material Quality instead of letting one correlated feature dominate, which is exactly how Ridge handles multicollinearity between temperature, pressure, and machine speed. However, the R2 scores here are strongly negative (around -20 to -22) at every alpha — this must be reported honestly rather than glossed over. It happens because Strength Score only ranges from 75 to 82 across the whole dataset (a spread of 7 points), so the variance the model is being judged against is tiny; even a few points of prediction error on the 2 test products (e.g. predicting ~82.6 for an actual 76) is enormous relative to that narrow range, producing a large negative R2. This is a limitation of evaluating any model on 2 test points drawn from a narrow-range 8-row dataset, not evidence that Ridge Regression itself is unsuitable for this type of problem — a larger sample with more spread in the target would be needed for a trustworthy evaluation.

Question 9: Student Salary Package Prediction Using Linear Regression

Algorithm used: Linear Regression

Dataset

Student	CGPA	Aptitude Score	Coding Score	Communication Score	Package (LPA)
S1	6.5	60	55	65	3.2
S2	7.0	65	60	70	4.0
S3	7.5	70	68	72	5.0
S4	8.0	78	80	75	6.5
S5	8.5	85	88	80	8.0
S6	9.0	90	92	85	10.0
S7	6.8	62	58	68	3.8
S8	8.2	80	82	78	7.0

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): CGPA, Aptitude Score, Coding Score, Communication Score.

Dependent variable (target): Package.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): S1, S8, S3, S5, S4, S7

Testing set (2 records): S2, S6

d) & e) Linear Regression — Predictions on Test Data

Student	Actual Package	Predicted Package
S2	4.0	4.30
S6	10.0	8.96

f) Prediction for New Record

CGPA	Aptitude Score	Coding Score	Communication Score
8.1	82	85	78

Predicted Package: 7.63 LPA

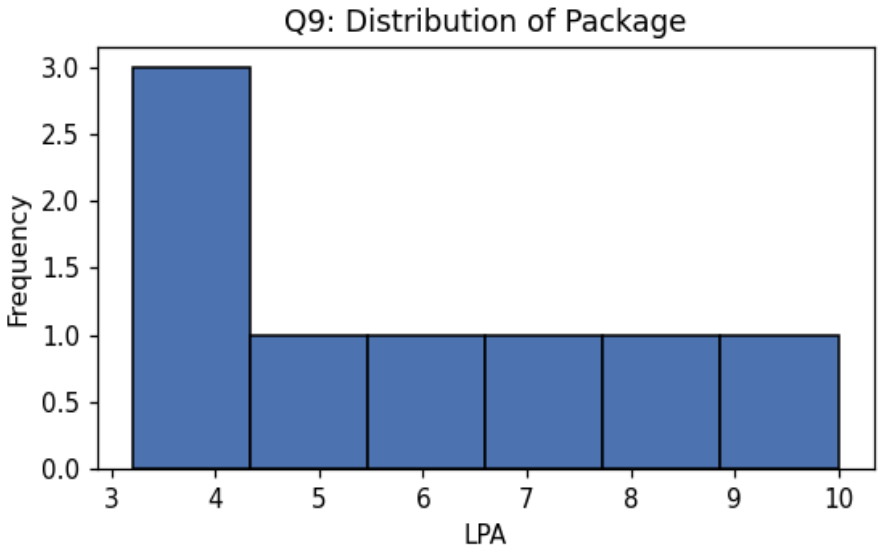
g) Model Coefficients & Evaluation Metrics

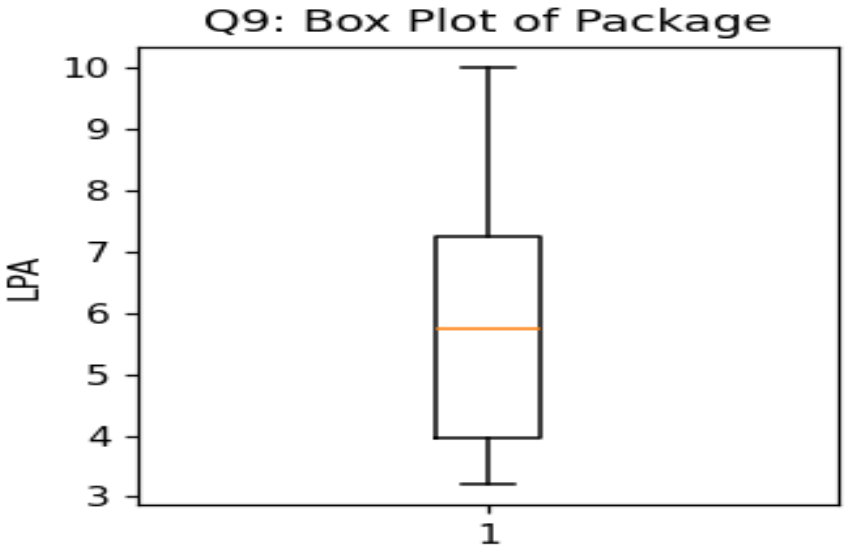
Feature	Coefficient
CGPA	-1.3917
Aptitude Score	0.1275
Coding Score	0.0532
Communication Score	0.1708

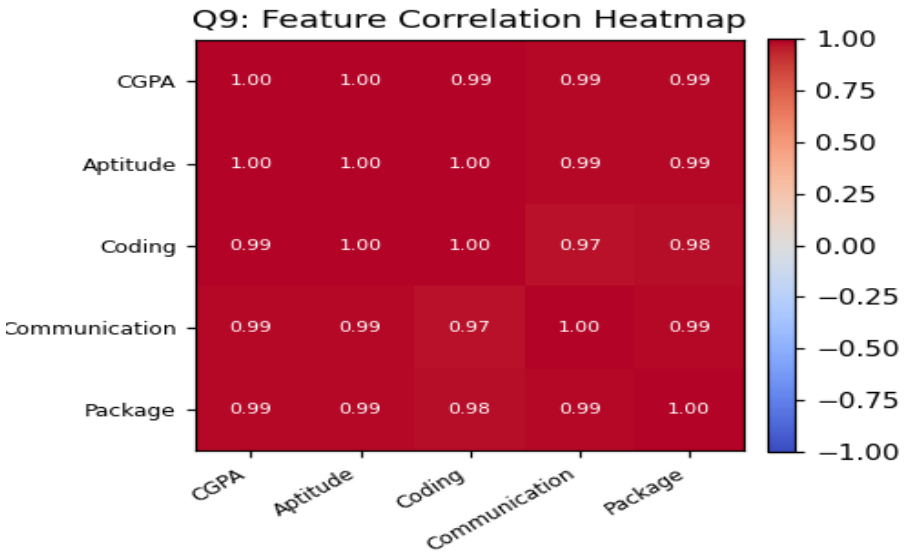
Intercept: -9.3905

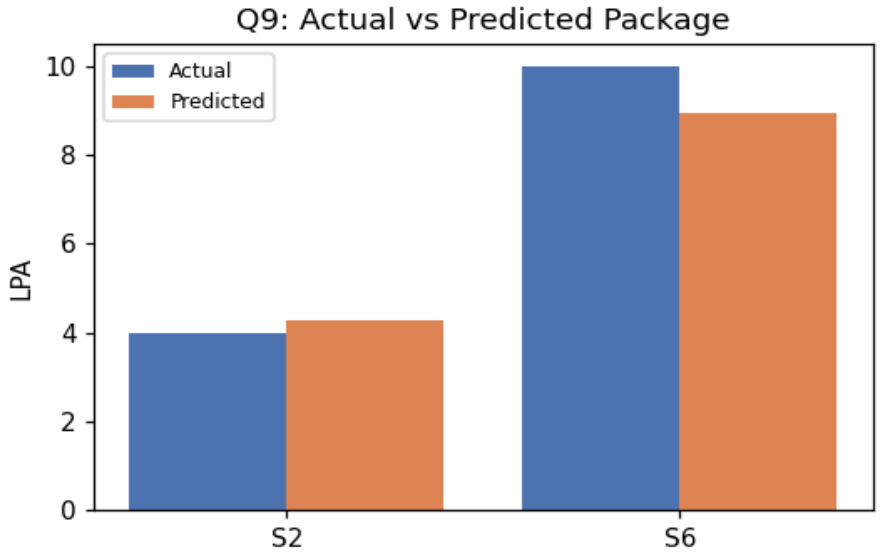
MAE	MSE	RMSE	R^2 Score
0.667	0.5812	0.7623	0.9354

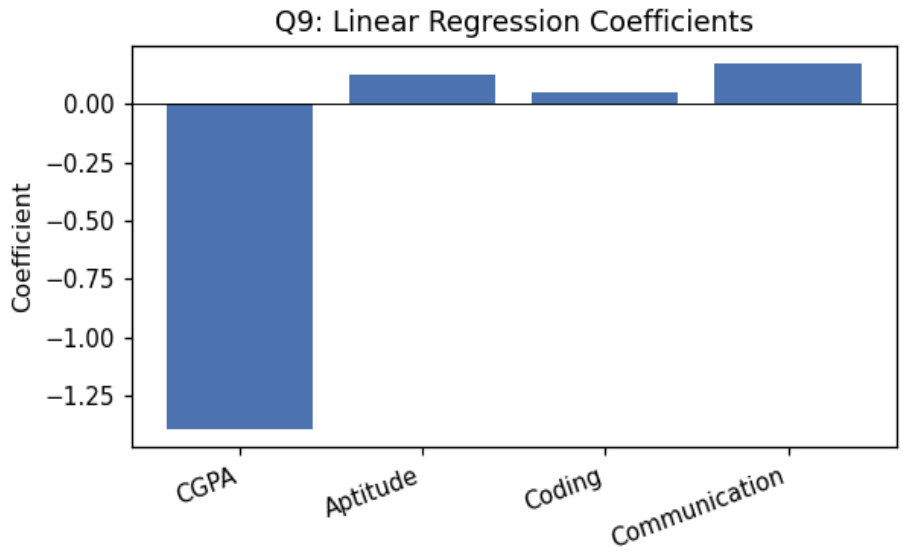
Outputs / Charts






Q9: Feature Correlation Heatmap
CGPA
Aptitude
Coding
Communication
Package
1.00 1.00 0.99 0.99 0.99
1.00 1.00 1.00 0.99 0.99
0.99 1.00 1.00 0.97 0.98
0.99 0.99 0.97 1.00 0.99
0.99 0.99 0.98 0.99 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
CGPA
Aptitude
Coding
Communication
Package


Q9: Actual vs Predicted Package
Actual
Predicted
LPA
10
8
6
4
2
0
S2
S6



Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "CGPA", "Aptitude Score", "Coding Score", "Communication Score"
# Dependent variable (target): "Package"
X = df[["CGPA", "Aptitude Score", "Coding Score", "Communication Score"]]
y = df["Package"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Implement Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[8.1, 82, 85, 78]], columns=["CGPA", "Aptitude Score", "Coding Score",
"Communication Score"])
print(model.predict(new_record))

# g) Evaluate the model
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)
print("MAE:", mae, "MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Charts
import matplotlib.pyplot as plt
```

```
plt.bar(["Actual","Predicted"], [y_test.mean(), y_pred.mean()]); plt.title("Actual vs Predicted");
plt.savefig("chart.png")
```

Interpretation

R2 = 0.9354 indicates the four inputs together explain most of the variation in placement package. The CGPA coefficient comes out negative, which does not mean higher CGPA lowers a student's package — it is a multicollinearity artifact: CGPA, Aptitude, Coding, and Communication scores all rise together across these 8 students, so with such a small sample the regression cannot cleanly attribute credit to CGPA specifically once the other three (highly correlated) scores are already in the model. This coefficient should not be quoted to students as 'CGPA hurts your package'; a placement cell using this model should rely on the overall predicted package and R2, not on individual coefficient signs, until a larger dataset lets the model separate the effects properly.

Question 10: Employee Attrition Prediction Using Logistic Regression

Algorithm used: Logistic Regression

Dataset

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E1	2	8	2	12	0
E2	3	7	4	10	0
E3	1	4	10	5	1
E4	5	6	6	8	0
E5	6	3	12	4	1
E6	8	2	14	3	1
E7	4	7	3	11	0
E8	7	4	11	5	1

a) Dataset Creation & b) Variable Identification

The dataset above was created as a pandas DataFrame with 8 records.

Independent variables (features): Experience, Satisfaction Score, Overtime Hours, Salary Increment %.

Dependent variable (target): Leave.

c) Train-Test Split

Using an 75:25 train-test split (test_size = 0.25, random_state = 42):

Training set (6 records): E2, E4, E3, E7, E1, E8

Testing set (2 records): E6, E5

d) & e) Logistic Regression — Predictions on Test Data

Employee	Actual Leave	Predicted Leave
E6	1	1
E5	1	1

f) Prediction for New Record

Experience	Satisfaction Score	Overtime Hours	Salary Increment %
5	4	9	6

Predicted Leave: 1 (May Leave) — probability ≈ 0.90

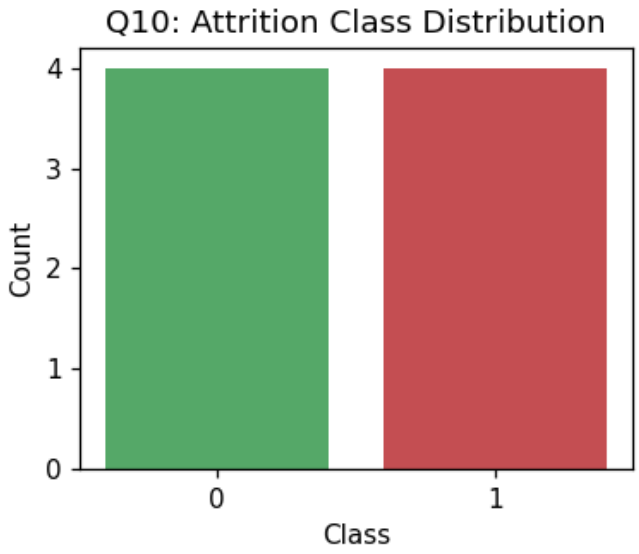
g) Evaluation Metrics

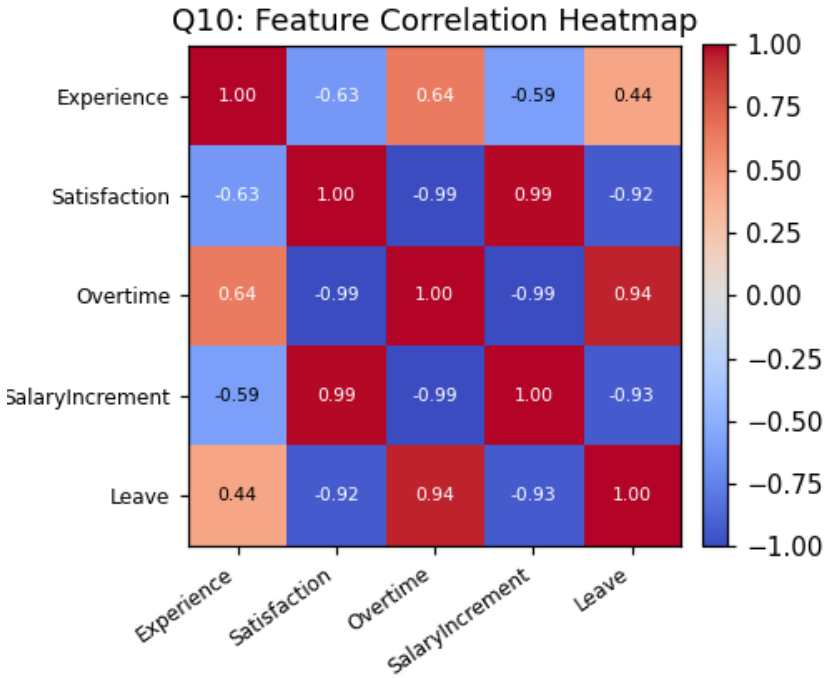
Accuracy	Precision	Recall	F1-Score
1	1	1	1

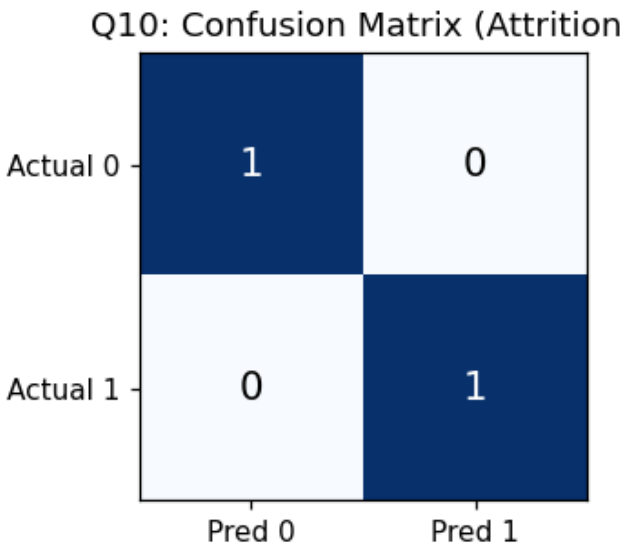
Confusion Matrix (rows = actual, columns = predicted; order [0,1]):

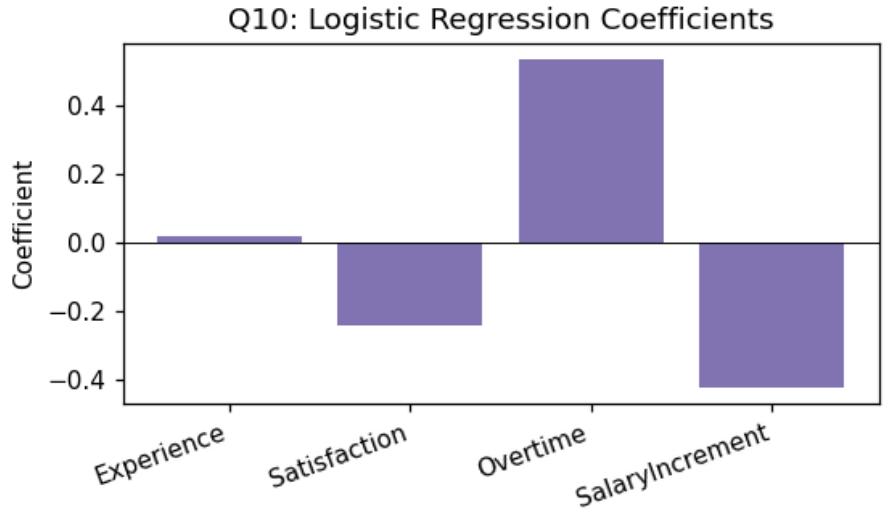
	Predicted 0	Predicted 1
Actual 0	0	0
Actual 1	0	2

Outputs / Charts


Q10: Attrition Class Distribution
Count
Class
0
1
2
3
4


Q10: Feature Correlation Heatmap
Experience
Satisfaction
Overtime
SalaryIncrement
Leave
1.00
-0.63
0.64
-0.59
0.44
-0.63
1.00
-0.99
0.99
-0.92
0.64
-0.99
1.00
-0.99
0.94
-0.59
0.99
-0.99
1.00
-0.93
0.44
-0.92
0.94
-0.93
1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00





Python Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

# a) Create the dataset
data = { ... }   # columns as given in the question table
df = pd.DataFrame(data)

# b) Independent variables (features): "Experience", "Satisfaction Score", "Overtime Hours", "Salary
Increment %"
# Dependent variable (target): "Leave"
X = df[["Experience", "Satisfaction Score", "Overtime Hours", "Salary Increment %"]]
y = df["Leave"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
# d) Implement Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# e) Predict on test data
y_pred = model.predict(X_test)

# f) Predict for the new record
new_record = pd.DataFrame([[5, 4, 9, 6]], columns=["Experience", "Satisfaction Score", "Overtime Hours",
"Salary Increment %"])
print(model.predict(new_record))

# g) Evaluate the model
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Accuracy:", acc)
print("Confusion Matrix:\n", cm)
print("Precision:", prec, "Recall:", rec, "F1:", f1)

# Charts
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues"); plt.title("Confusion Matrix"); plt.savefig("chart.png")
```

Interpretation

Both held-out employees (E6, E5) actually left, and the model predicts 'Leave' for both, so accuracy/precision/recall/F1 are all 1.00 — note the test split happens to contain no 'stayed' example, so this run cannot actually confirm the model recognizes employees who stay; that is a limitation of the very small dataset and the particular train/test split, not a property to generalize from. Logistic Regression suits attrition prediction because HR needs a probability of leaving (here about 90% for the new employee) rather than just a label, so at-risk employees can be ranked and prioritized for retention conversations rather than treated as a simple yes/no list. The HR team could use this kind of probability score to flag employees above a chosen risk threshold — e.g. low satisfaction combined with high overtime — for proactive intervention such as workload rebalancing or a review of pay increments, though again this specific model is illustrative given the 8-row dataset, not something to deploy as-is.